Technical Specification: Latent Social Dynamics in Public Goods Games via PLRNN State-Space Models

Project Codename: DSC-EPGG

Version: 3.2 **Status:** Pre-implementation (handoff-ready)

---

0. Executive Summary

We fit a Piecewise-Linear Recurrent Neural Network (PLRNN) state-space model to multi-agent behavioral trajectories from a repeated Extended Public Goods Game (EPGG) with incentive uncertainty and emergent communication. The PLRNN reconstructs interpretable low-dimensional latent dynamics — cooperation regimes, trust-like signal weighting, communication conventions — that go beyond what a Bayesian filter over the hidden incentive factor can capture. The primary result is a dynamical-systems analysis showing how communication and uncertainty reshape the attractor landscape of cooperation.

---

1. Environment Specification

1.1 Base Game: Extended Public Goods Game (EPGG)

Source codebase: https://github.com/nicoleorzan/marl-emecom **Framework:** PettingZoo (parallel API)

Per-round mechanics:

- N agents (default $N = 4$), each with fixed endowment c_i (default $c = 4$ coins).
- Each agent chooses action $a_i \in \{C, D\}$ (Cooperate = invest full endowment; Defect = keep endowment).
- Payoff function:

$$u_i(\mathbf{a}, f, \mathbf{c}) = \frac{1}{N} \sum_{j=1}^{N} c_j \cdot \mathbb{1}[a_j = C] \cdot f + c_i \cdot (1 - \mathbb{1}[a_i = C])$$

- The multiplication factor f determines the game regime:
  - $f \leq 1$: **Competitive** — defection is dominant strategy
  - $1 < f \leq N$: **Mixed-motive** — cooperation is Pareto optimal but defection is weakly dominant
  - $f > N$: **Cooperative** — cooperation is dominant strategy

1.2 Sticky-EPGG Extension (Markov Incentive Dynamics)

Purpose: Create temporal structure that justifies dynamical modeling.

Change: Replace i.i.d. sampling of f_t with a Markov chain:

$$P(f_{t+1} = f_t) = 1 - \rho$$
$$P(f_{t+1} = f' \mid f_{t+1} \neq f_t) = \text{Uniform}(F \setminus \{f_t\})$$

Parameters: - $F = \{0.5, 1.5, 2.5, 3.5, 5.0\}$ covering competitive, mixed, and cooperative regimes. - $\rho = 0.05$ (default; expected regime duration ~20 rounds). Sweep $\rho \in \{0.02, 0.05, 0.1, 0.2\}$.

Implementation: Store `current_f` as environment state. At each `step()`, draw $u \sim \text{Uniform}(0, 1)$; if $u < \rho$, resample from $F \setminus \{f_t\}$.

Regime identifiability audit (must run before main experiments): Compute: how many rounds does a Bayes-optimal observer need to concentrate $P(f_t)$ above 0.9 after a regime switch, given σ_i and the observation model? If ≤ 2 rounds, increase σ or reduce $|F|$.

1.3 Multi-Step Episodes (Repeated Sessions)

Change: `num_game_iterations = T` where $T = 100$.

Required code changes (non-trivial):

The current `pgg_parallel_v0.py` has the following problems:

1. **`step()` only returns observations in the `env_done` branch.** In the non-done branch, `observations` is undefined. Fix: call `observe()` every step; return updated observations regardless of `done`.
2. **`current_multiplier` only set in `reset()`.** Fix: implement Markov f_t update inside `step()`.
3. **Done-branch returns dummy zeros.** Fix: return actual observations.
4. **Return signature is 4-tuple.** Pin to old PettingZoo version (see §10).
5. **`observation_space()` returns `Discrete(obs_space_size)`** but `observe()` returns a float tensor. Fix: change `observation_space()` to return `Box(low=-inf, high=inf, shape=(obs_dim,), dtype=np.float32)`.

Unit tests required: - `step()` returns valid observations for all T steps. - f_t changes according to Markov dynamics. - Observations update each round. - Episode terminates after exactly T steps. - Rewards match payoff formula for current f_t. - Observation space dtype/shape matches actual observations.

1.4 Canonical Round Timing (REFERENCE: all implementations must follow this order)

Round t protocol (steps 1–8 are atomic and sequential):

1. **State:** Environment holds f_t.
2. **Observe:** Env generates o_t containing $\hat{f}_{i,t} \sim \mathcal{N}(f_t, \sigma_i^2)$, endowment c_i.
3. **Wrapper augments:** `ObservationWrapper.build_obs()` appends history features (k_{t-1}/N, $\tilde{a}_{i,t-1}$, EWMA) to produce Set A observation.
4. **Messages:** Communicating agents send $m_{i,t}$ based on augmented o_t. Dropout applied. Listening agents' observations rebuilt with messages.
5. **Actions:** Agents choose intended actions $a_{i,t}$ from policy $\pi(a \mid o_t^{\text{aug}})$.
6. **Tremble:** Env applies trembling hand $\rightarrow$ executed actions $\tilde{a}_{i,t}$.
7. **Rewards:** $r_{i,t}$ computed from $\tilde{a}_t$ and f_t (the same f used in step 2).
8. **Transition:** Env draws f_{t+1} via Markov hazard ρ. Next round starts at step 1 with f_{t+1}.

Derived quantities: - $\pi_t := P(f_t = f \mid \hat{f}_{1:N,1:t})$ — posterior using observations up through step 2 of round t. - $k_t := \sum_i \tilde{a}_{i,t}$ — cooperation count from executed actions at round t. - History features at round t (step 3) use k_{t-1}, $\tilde{a}_{i,t-1}$ — one round lagged.

1.5 Information Sets (CRITICAL: Three Distinct Sets)

This separation must be maintained rigorously.

Set A: What agents observe BEFORE acting at round t

Feature	Dim	Description
$\hat{f}_{i,t}$	1	Noisy observation of current f_t
c_i	1	Own endowment (constant)
k_{t-1}/N	1	Fraction of cooperators last round (0 at $t = 0$)
Own last executed action $\tilde{a}_{i,t-1}$	1	Binary (what actually happened, not what was intended)
EWMA cooperation $\bar{k}_t$	1	Decay $\lambda = 0.9$
Received messages $m_{j,t}$	$\lvert Z\rvert \times V$	One-hot (if comm enabled)

Total (no comm): 5. **With comm ($|Z| = 3$, $V = 2$):** 11.

EXCLUDED from agent observations (Set A):

Feature	Why excluded
W_{t-1} (group welfare)	Combined with k_{t-1}, reveals f_t from the payoff equation
$r_{i,t-1}$ (own reward)	Leaks f_t when combined with k_{t-1} and own action
True f_t	Ground truth; never shown
Flip labels	Simulation internal

Optional coarsened reward feedback: If Dynamic Richness Checklist fails, provide $\tilde{r}_{i,t-1} \in$ {low, med, high} (terciles). Start without it.

Set B: What agents receive AFTER acting (for learning only)

- Own reward $r_{i,t}$: used in PPO/REINFORCE return computation. **Not** fed back as observation input at $t+1$.

Set C: What the PLRNN / analysis pipeline observes Everything in Set A, plus: f_t, regime label, flip labels, intended actions, exact rewards, group welfare.

Set D: What the PLRNN receives as INPUT vs predicts as OUTPUT This must be explicit to prevent the PLRNN from re-inferring f_t through rewards.

Role	Variables
PLRNN inputs (y_t^{in})	Executed actions $\tilde{a}_{1:N,t}$ + messages $m_{1:N,t}$ (if comm)
PLRNN exogenous input (u_t)	Full Bayesian posterior $\pi_t \in \mathbb{R}^{\lvert F \rvert}$
PLRNN emission targets (y_t^{out})	$\tilde{a}_{1:N,t}$ + $m_{1:N,t}$ + $r_{1:N,t}$ (all at time t)

All observation components at time t are grouped together. The emission $p(y_t^{\text{out}} \mid z_t)$ predicts actions, messages, and rewards that all occur within the same round. "Predict next step" means $p(y_{t+1} \mid z_t)$, where the model autoregressively rolls z forward via the transition.

Why this indexing: Rewards at round t are causally determined by $\tilde{a}_t$ and f_t, so grouping them at the same timestep respects causal structure. The mixed-index alternative (r_t but $\tilde{a}_{t+1}$) is defensible but creates unnecessary implementation complexity for no scientific gain.

Encoder input is a subset of y_t^{out}: only $(\tilde{a}_{1:N,t}, m_{1:N,t})$. Rewards are emission targets only — they are never fed to the encoder, preventing the PLRNN from re-inferring f_t through the payoff structure. All baselines use the same input/output split for fair comparison.

Note on k_{t-1} as social sensor: Once agents learn regime-conditioned policies, k_{t-1} carries information about f_t through policy-mediated channels. This is by design — π_t is intentionally a **private-signal-only** posterior (computed from $\hat{f}$ observations only). The PLRNN's ability to use social signals (k, messages) to improve predictions beyond π_t is the core claim. As a robustness check, we also compute an **augmented posterior** $\pi_t^{(+k)}$ that incorporates k_{t-1} via a learned likelihood $p(k \mid f)$ estimated from data, and verify that the PLRNN still adds predictive value over this stronger baseline (§6.2).

1.6 Architecture: Trainer-Side Observation Wrapper

Communication is orchestrated in the training loop, not the environment. We extend this pattern.

- **The environment** handles: game mechanics, f_t dynamics, noisy $\hat{f}$, reward computation, termination.
- `ObservationWrapper` (trainer-side) constructs augmented observations from env output + history buffers + received messages.

The wrapper maintains: - Rolling buffer of last-round cooperation counts. - EWMA cooperation tracker. - Last-round action per agent. - Received messages.

1.7 Incentive Uncertainty

$$\hat{f}_{i,t} = f_t + \epsilon_{i,t}, \quad \epsilon_{i,t} \sim \mathcal{N}(0, \sigma_i^2)$$

Remove observation clamping from the env's `observe()`. Raw Gaussian noise; no clipping. This keeps the Bayesian baseline's Gaussian likelihood exact.

Conditions:

Condition	σ_i values
None	$\sigma_i = 0$ for all i
Symmetric	$\sigma_i = 0.5$ for all i
Asymmetric	$\sigma_i = 0$ for $i \leq N/2$; $\sigma_i = 0.5$ for $i > N/2$

1.8 Emergent Communication Channel

Subset $Z \subseteq \{1, \ldots, N\}$ can send $m_{i,t} \in \{1, \ldots, V\}$ before acting.

Vocabulary: $V = 2$ (default), $V = 5$ (ablation).

Auxiliary losses (positive signaling, positive listening): keep from marl-emecom.

Message dropout: Applied in the trainer-side wrapper. With probability $p_{\text{drop}} = 0.1$, replace each received message with a sample from the **per-sender marginal** (running average of that sender's message frequencies over the current training batch).

1.9 Trembling-Hand Action Noise

$$\tilde{a}_{i,t} = \begin{cases} 1 - a_{i,t} & \text{with prob } \epsilon_{\text{tremble}} \\ a_{i,t} & \text{otherwise} \end{cases}$$

Applied env-side in `step()`, after receiving agent actions.

Logging distinction: - **Training buffer** stores the **intended** action $a_{i,t}$ (for policy gradient — the agent's actual sample from its policy). - **Data logger** stores both intended $a_{i,t}$ and executed $\tilde{a}_{i,t}$. - **Payoffs** computed from $\tilde{a}$. - **Other agents observe** $\tilde{a}$ in subsequent rounds.

Default: $\epsilon_{\text{tremble}} = 0.05$.

1.10 Hard Configuration Requirements

- **`gmm_ = False`** in all agent configs (existing GMM logic assumes scalar obs; will break with vectors).
- **`normalize_obs = False`** (current normalization divides by scalar max; incompatible with multi-feature vectors).
- **`observation_space`** must return `Box`, not `Discrete`.

---

2. Agent Training

2.1 Architecture

Policy network: Feed-forward MLP. - Input: observation vector (§1.5 Set A) - Hidden: 2 layers, 64 units, ReLU - Output (comm policy): softmax over V messages - Output (action policy): sigmoid → Bernoulli over $\{C, D\}$

No recurrent layers.

2.2 Roles, Identity & Parameter Sharing

No parameter sharing. Each agent has its own policy network and value network. Rationale: asymmetric conditions give agents structurally different information ($\sigma_i = 0$ vs $\sigma_i = 0.5$); shared parameters would force a single policy to handle both, collapsing the asymmetry that drives exploitation dynamics.

Fixed role assignment across all episodes and seeds: - Agents $\{0, 1\}$: informed ($\sigma_i = 0$ in asymmetric conditions; $\sigma_i = 0.5$ in symmetric). - Agents $\{2, 3\}$: uncertain ($\sigma_i = 0.5$ in asymmetric conditions; same in symmetric).

Communication roles (Z): - **Comm-on conditions:** $Z = \{0, 1, 2, 3\}$ (all agents send). This is the default. - **Ablation (optional):** $Z = \{0, 1\}$ (only informed agents send). Run only if exploitation dynamics are absent with $Z =$ all.

PLRNN per-agent emission heads (α_i, V_i, μ_i) are indexed by agent identity, which is fixed. Cross-seed comparisons are valid because agent 0 always has the same structural role.

2.3 Training Algorithm

Default: PPO with Generalised Advantage Estimation (GAE).

The existing marl-emecom REINFORCE implementation multiplies each step's logprob by the *immediate* reward — there is no return-to-go computation. This is correct for 1-step episodes but kills credit assignment in $T = 100$ sessions: agents cannot learn history-dependent strategies because today's action affects future rounds through history features, not just today's reward.

PPO specification: - Collect full T-step trajectories per episode. - Compute GAE advantages with $\gamma = 0.99$, $\lambda_{\text{GAE}} = 0.95$. - Value function $V_\phi(o_{i,t})$: same MLP architecture, separate parameters. - PPO clip ratio $\epsilon_{\text{clip}} = 0.2$. - $K = 4$ epochs per batch of trajectories. - Mini-batch size: 32 episodes.

Note on existing code: The training loop in `train_given_params.py` contains `agent.return_episode =+ rewards[...]` (likely should be `+=`). This must be fixed or bypassed in the new training loop.

Fallback: If PPO introduces instabilities, try A2C (simpler, still uses returns). Do NOT fall back to myopic REINFORCE — it fundamentally cannot train the history-dependent strategies this project requires.

Communication under PPO (joint policy, single advantage):

Within each round t, communicating agents make two sequential decisions: message $m_{i,t}$, then action $a_{i,t}$ (after receiving others' messages). Both share the same round-level advantage $\hat{A}_t$.

- **PPO surrogate uses joint logprob:** $\log \pi_\theta(m_{i,t} \mid o_t) + \log \pi_\theta(a_{i,t} \mid o_t, m_{-i,t})$.
- **Clipping applies to the joint ratio:** $r_t(\theta) = \frac{\pi_\theta(m_{i,t}) \cdot \pi_\theta(a_{i,t})}{\pi_{\theta_{\text{old}}}(m_{i,t}) \cdot \pi_{\theta_{\text{old}}}(a_{i,t})}$.
- **Auxiliary losses** (positive signaling, positive listening from marl-emecom) are added with fixed coefficient $\lambda_{\text{aux}} = 0.1$.
- **Entropy bonus** applies to both message and action distributions independently.

This is the simplest correct design. A two-substep MDP (message at substep 1, action at substep 2) would give cleaner credit assignment but doubles the buffer length and complicates GAE. Not worth it for the MVP.

2.4 Training Curriculum

1. **Phase 1 (warmup):** Train on fixed f values separately. ~50k episodes per value.
2. **Phase 2 (Sticky-EPGG):** Train on Markov-switching f_t. ~200k episodes.
3. **Phase 3 (add comm):** Enable communication. Train from Phase 2 checkpoint. ~200k episodes.

2.5 Dynamic Richness Checklist

After training, before PLRNN fitting:

- ☐ Cooperation rate differs across f regimes.
- ☐ History terms significant: regress $a_{i,t}$ on $(\hat{f}_{i,t}, k_{t-1}, k_{t-2}, k_{t-3})$; lagged terms have nonzero coefficients.
- ☐ Visible within-session transitions (non-monotonic rolling cooperation).
- ☐ Communication effects (if enabled): $\mathrm{MI}(m; f_t) > 0$ and $\mathrm{MI}(m; a) > 0$.
- ☐ Agent heterogeneity: variance of cooperation across agents is non-negligible.

Escalation order if fewer than 3 pass:

1. Increase ρ (more regime switches)
2. Increase σ (more observation noise)
3. **Add lag features to Set A:** k_{t-2}/N, k_{t-3}/N, and per-agent defection count in last 5 rounds. Cheapest intervention that meaningfully expands representable strategies without recurrence. (Do NOT add recurrent policies: if dynamics live inside the policy, the PLRNN story is muddied.)
4. Increase $\epsilon_{\text{tremble}}$
5. Increase p_{drop}
6. Increase V (vocabulary size)
7. Add endogenous f feedback (§7.1)
8. Add agent heterogeneity (§7.2)

---

3. Data Logging Specification

3.1 Per-Round Logging Vector

Ground truth (Set C, never shown to agents):

Field	Type
`f_t`	float
`regime_t`	categorical
`flip_i_t`	binary × N
`a_i_t_intended`	binary × N

Observable:

Field	Type
`f_hat_i_t`	float × N
`m_i_t`	int × N (if comm)
`a_tilde_i_t`	binary × N
`r_i_t`	float × N
`k_t`	int
`W_t`	float

3.2 Storage

- **Collection phase:** One `.npz` per session. Naming: `data_{condition}_{seed}_{session_id}.npz`.
- **Pre-training consolidation (required step):** Before PLRNN fitting, consolidate into one file per (condition, seed) as arrays shaped `(num_sessions, T, feature_dim)`. Use HDF5 or a single large `.npz`. This prevents I/O bottlenecks from reading thousands of small files during training.

3.3 Dataset Size

Parameter	Value
Sessions per condition per seed	500
Seeds per condition	5
Rounds per session (T)	100
Total sessions per condition	2500
Priority conditions	6 (see §5)

---

4. PLRNN State-Space Model

4.1 Implementation

Custom PyTorch. Existing PLRNN toolkits (e.g., dendPLRNN, GPL-3.0 licensed) assume Gaussian observations. Our factored emission (Bernoulli + Categorical + Gaussian) requires custom code. We reuse the PLRNN **transition parameterization** and initialization heuristics from the Durstewitz literature but implement our own training pipeline.

4.2 Training: Stochastic Variational Inference

- **Recognition network (encoder):** Bidirectional GRU mapping observations to approximate posterior $q_\phi(z_{1:T} \mid y^{\text{in}}_{1:T})$, parameterized as diagonal Gaussian at each t.
- **Generative model:** PLRNN transition + factored emission.
- **Objective:** ELBO $= \mathbb{E}_q[\sum_t \log p(y^{\text{out}}_t \mid z_t)] - D_{\text{KL}}(q(z_{1:T}) \| p(z_{1:T}))$.

CRITICAL: Causal vs. smoothing inference.

The bidirectional encoder produces *smoothed* posteriors $q(z_t \mid y_{1:T})$ — these use future information. This is fine for model fitting (best ELBO). But for **RQ3 early warning evaluation**, using smoothed z_t would inflate detection lead times ("peeking into the future").

Solution: Implement **two inference paths:** 1. **Smoothing (biGRU):** Used for model fitting and for RQ1/RQ2 evaluation (where we care about reconstruction quality, not online prediction). 2. **Causal (forward-only GRU):** Used for all RQ3 evaluation. Trained to approximate the smoothing posterior via a distillation loss: $D_{\text{KL}}(q_{\text{causal}}(z_t \mid y_{1:t}) \| q_{\text{smooth}}(z_t \mid y_{1:T}))$ averaged over t. Or: train a separate forward-only encoder from scratch.

All early warning results in the paper must use the causal encoder. This must be stated explicitly in the methods section.

Alternative (simpler): Train with a forward-only encoder from the start. Slightly worse ELBO but avoids the two-encoder complexity. Acceptable if the ELBO gap is small.

4.3 Latent Dynamics

$$z_{t+1} = Az_t + W\,\text{ReLU}(z_t) + Bu_t + \eta_t, \quad \eta_t \sim \mathcal{N}(0, Q)$$

- $z_t \in \mathbb{R}^d$, default $d = 10$, sweep $d \in \{5, 8, 10, 15\}$. Cap at $d = 15$ for attractor analysis.
- $A \in \mathbb{R}^{d\times d}$: drift/persistence.
- $W \in \mathbb{R}^{d\times d}$: nonlinear interaction.
- $B \in \mathbb{R}^{d\times |F|}$: input gain.
- Q: diagonal process noise (learned).

4.4 Exogenous Input: Full Bayesian Posterior

$$u_t = \pi_t \in \mathbb{R}^{|F|}$$

where $\pi_t(f) = P(f_t = f \mid \hat{f}_{1:N,1:t})$, computed via exact HMM forward pass.

Observation model in filter must match the environment: no clamping, Gaussian likelihood $\hat{f}_{i,t} \sim \mathcal{N}(f, \sigma_i^2)$.

π_t **is a private-signal-only posterior** — computed from $\hat{f}$ observations only, not from behavioral signals like k_{t-1} or messages. This is intentional: the PLRNN's ability to extract additional information from social signals is part of the "beyond f_t" claim.

Role-specific posteriors (log and use for interpretability):

In asymmetric conditions ($\sigma_i = 0$ for informed agents), the central π_t using all N agents' observations becomes near-delta almost immediately, erasing the belief asymmetry that drives trust/exploitation dynamics. To preserve this for analysis:

- **Always log:** $\pi_t^{(\text{inf})}$ (posterior from informed agents' $\hat{f}$ only) and $\pi_t^{(\text{unc})}$ (posterior from uncertain agents' $\hat{f}$ only).
- **Primary PLRNN input remains π_t (central).** This is the strongest possible regime input, making the "beyond f_t" claim hardest to dismiss. If we used $\pi_t^{(\text{unc})}$, the PLRNN might win by doing better regime inference, which is a weaker claim.
- **Interpretability run (asymmetric conditions only):** Re-fit PLRNN with $u_t = \pi_t^{(\text{unc})}$. If the PLRNN now captures more belief-gap dynamics in its latent space, report this as evidence that the latent encodes social cognition beyond regime identity.

4.5 PLRNN Input/Output Specification

Component	Variables	Notes
Encoder input (y_t^{in})	$\tilde{a}_{1:N,t}$, $m_{1:N,t}$ (if comm)	Actions and messages only. **No rewards.**
Exogenous (u_t)	$\pi_t \in \mathbb{R}^{\lvert F\rvert}$	Full Bayesian posterior
Emission targets (y_t^{out})	$\tilde{a}_{1:N,t}$, $m_{1:N,t}$, $r_{1:N,t}$	All at time t

All components indexed at time t. "Predict next step" = generate z_{t+1} via transition, then emit $p(y_{t+1}^{\text{out}} \mid z_{t+1})$.

All baselines use the same input/output split. This ensures fair comparison.

4.6 Observation Model (Factored Emission)

$$p(y_t^{\text{out}} \mid z_t) = \prod_{i=1}^{N} p(\tilde{a}_{i,t} \mid z_t) \cdot \prod_{i \in Z} p(m_{i,t} \mid z_t) \cdot \prod_{i=1}^{N} p(r_{i,t} \mid z_t)$$

- $p(\tilde{a}_{i,t} = C \mid z_t) = \sigma(\alpha_i^\top z_t + b_i^{(a)})$ — Bernoulli
- $p(m_{i,t} \mid z_t) = \text{softmax}(V_i z_t + b_i^{(m)})$ — Categorical
- $p(r_{i,t} \mid z_t) = \mathcal{N}(\mu_i^\top z_t + b_i^{(r)}, \sigma_r^2)$ — Gaussian, with $\sigma_r \geq 0.1$ (floored)

Reward emission guard: Because rewards are near-deterministic given $(\tilde{a}_t, f_t)$, and f_t is strongly constrained by $u_t = \pi_t$, the Gaussian reward term can dominate the ELBO if $\sigma_r \to 0$, causing the latent to encode reward geometry rather than social dynamics. Mitigations (apply at least one): - Floor $\sigma_r \geq 0.1$ (default). - Downweight reward log-likelihood by coefficient $\beta_r = 0.1$ in the ELBO. - If both fail, remove reward from ELBO entirely and evaluate reward prediction post-hoc only.

4.7 Optional Structural Constraints

- **Block-sparse B:** u_t feeds only into first $d_f = |F|$ dimensions.
- **Orthogonality regulariser:** Linear probe predicting f_t from $z_t^{(d_f+1:d)}$ with gradient reversal.

Optional. The core "beyond f_t" test works without them.

4.8 Training Details

- Optimizer: Adam, lr $= 10^{-3}$, cosine decay.
- Batch: 32 sessions.
- Epochs: 200, early stopping on held-out ELBO.
- L2 on W: $\lambda = 10^{-4}$.
- **10 random initialisations** per condition; select top 3 by validation ELBO; report attractor analysis stability across top 3.

SVAE anti-collapse measures (required):

A common SVAE failure mode: the encoder becomes strong, reconstruction looks good, but the generative transition (A, W, B) learns nothing (posterior collapse / encoder explains away dynamics). If this happens, fixed-point and Jacobian analysis is meaningless because it analyzes a transition not responsible for modeling the data.

- **KL annealing:** Warm up the KL weight from 0 to 1 over the first 20 epochs.
- **Free bits:** Enforce minimum KL of at least 0.1 nats per latent dimension to prevent total collapse.
- **Prior predictive sanity check (required before attractor analysis):** Simulate 100 sessions from the PLRNN generative model given u_t sequences from evaluation data (sample z_0, roll forward through transition, sample emissions). Verify generated sequences reproduce coarse statistics: cooperation rate distribution (mean within 0.1 of data), message entropy distribution, reward variance. If prior predictive is poor while reconstruction is good, the transition has not learned the dynamics. Do not proceed to attractor analysis.
- **Teacher-forced vs open-loop gap:** Report 1-step predictive log-likelihood using (a) posterior z_t from encoder and (b) z_t from generative rollout. If the gap exceeds 50 percent of the teacher-forced score, the transition is too weak.

4.9 PLRNN Variants

Variant	When to use
Standard PLRNN	Default
shPLRNN	If standard fits poorly at $d \leq 15$
dendPLRNN-style	If piecewise-linear regions too coarse

---

5. Experimental Design

5.1 Condition Matrix

#	Comm	Uncertainty	Tremble	Role
1	on	symmetric	0.05	Full model
2	off	symmetric	0.05	Comm ablation
3	on	asymmetric	0.05	Exploitation
4	off	asymmetric	0.05	Asymmetry baseline
5	on	symmetric	0.00	Tremble ablation
6	off	none	0.00	Minimal control

MVP: Conditions 1 and 2 only. Expand after pipeline verified.

5.2 Runs

- 5 seeds per condition, 500 evaluation sessions per seed.
- 70/15/15 train/val/test split for PLRNN.

---

6. Evaluation and Analysis

6.1 Research Questions

RQ1a (Sanity / ablation): Does the Bayesian filter correctly track f_t (passes identifiability audit, matches simulated regime distribution)? In a *no-u_t ablation* (PLRNN receives u_t = uniform at all times), does PLRNN fail to outperform the HMM at regime decoding? (If it does outperform, the latent is re-inferring f_t from behavioral signals — interesting but a different claim.)

RQ1b (Core): Conditioning on $u_t = \pi_t$, does PLRNN improve action/message/reward prediction over all baselines at multi-step horizons?

RQ2 (Primary): Does communication reshape the attractor landscape (number, stability, basins of fixed points)?

RQ3 (Application): Can eigenvalue-based monitoring (using **causal** inference) detect cooperation collapse earlier than behavioral indicators?

6.2 Evaluation Protocol for RQ1b

CRITICAL: Inference mode for RQ1b.

Primary RQ1b results use **causal (forward-only) inference** for all models. The PLRNN uses the causal encoder $q(z_t \mid y_{1:t})$; the GRU baseline is naturally causal; the linear SSM uses filtering; Bayesian baselines are causal by construction. This ensures a fair forecasting comparison.

Smoothed (biGRU) PLRNN results may be reported as a **secondary upper bound** labeled explicitly as "offline reconstruction," not "prediction." If the gap between causal and smoothed is large, that itself is informative (it quantifies how much future context helps).

All models receive $u_t = \pi_t$ and past actions + messages as input. They predict next actions, messages, rewards.

Models:

Model	Description
Bayesian filter only	Empirical regime-cooperation lookup
Bayesian + ToM-0	+ Grid-posterior partner cooperativeness (§6.3.2)
Bayesian + msg reliability (regime)	+ Dirichlet-Categorical sender model, predicts via ToM-0 mechanism (§6.3.3)
Linear SSM (with u_t)	Linear dynamics
Bayesian + msg intention	+ Dirichlet-Categorical $m \mid a^{\text{intended}}$, predicts via ToM-0 (S6.3.4)
GRU (with u_t, matched params)	Black-box nonlinear
PLRNN-SSM (with u_t)	Our model

Metrics: - Action: AUC-ROC for $\tilde{a}_{i,t} = C$. - Message: top-1 accuracy, cross-entropy. - Reward: RMSE, calibration. - Multi-step: $H \in \{1, 5, 10, 20\}$. **Longer horizons are the key test.**

Multi-step evaluation modes (BOTH required for $H > 1$):

All models take exogenous input $u_t = \pi_t$. At horizons $H > 1$, we must specify what happens with $u_{t+1}, \ldots, u_{t+H}$:

1. **Teacher-forced (conditional):** At each future step $t+k$, feed the *realised* u_{t+k} from the dataset (computed from actual $\hat{f}$ observations at that time). This tests: "given the true regime beliefs, how well does the model predict social dynamics?"
2. **Open-loop (forecasting):** Feed only u_t. Propagate forward using the known Markov transition: $u_{t+k|t} = u_t P^k$ where P is the regime transition matrix. No future $\hat{f}$ observations. This tests: "can the model genuinely forecast?"

Report both. If only teacher-forced is reported at long horizons, a reviewer can fairly argue the model was conditioning on future inputs, not forecasting. The gap between modes is itself informative — it quantifies how much PLRNN predictions depend on regime input vs learned internal dynamics.

Oracle-regime control (robustness check): Run the same evaluation with $u_t = \delta_{f_t}$ (true regime as one-hot vector) instead of π_t. If PLRNN still improves over baselines at $H \geq 5$ when the true regime is known, the gain is definitively *not* from better regime inference. This is a cheap addition that buys significant reviewer robustness.

Augmented-posterior robustness check: Compute $\pi_t^{(+k)}$ — a posterior that incorporates k_{t-1} via a learned likelihood $p(k \mid f)$ estimated from training data. Re-run Bayesian baselines with $u_t = \pi_t^{(+k)}$. If PLRNN still wins, it captures dynamics beyond what k-augmented rational inference provides.

6.3 Bayesian Baseline Suite

6.3.1 Baseline 1: HMM Filter for f_t

- State space: $f \in F$ (5 values).
- Transition: known Markov chain $P(f_{t+1} \mid f_t)$.
- Observation: $\hat{f}_{i,t} \sim \mathcal{N}(f, \sigma_i^2)$, no clamping.
- Exact forward algorithm.
- **Prediction:** For each regime, store the empirical mean cooperation rate from training data. Predict as posterior-weighted average: $P(\tilde{a}_{j,t+1} = C) = \sum_f \pi_t(f) \cdot \bar{c}_{j,f}$ where $\bar{c}_{j,f}$ is agent j's average cooperation rate under regime f.

6.3.2 Baseline 2: Bayesian + Partner Cooperativeness (ToM-0) For each agent j and regime bin $r \in \{\text{comp, mixed, coop}\}$, maintain a posterior over cooperation propensity $\theta_{j,r} \in [0, 1]$.

Tremble-aware update (grid posterior):

The likelihood with trembling hand is:

$$P(\tilde{a}_j = C \mid \theta_{j,r}) = \theta_{j,r}(1-\epsilon) + (1-\theta_{j,r})\epsilon$$

This is NOT conjugate with a Beta prior. Use a **grid posterior**: - Discretise $\theta \in \{0, 0.01, 0.02, \ldots, 1.0\}$ (101 points). - Prior: uniform. - Update: multiply grid by likelihood, renormalise. - Prediction: $P(\tilde{a}_{j,t+1} = C) = \sum_\theta P(\tilde{a} = C \mid \theta) \cdot p(\theta \mid \mathcal{H}_t)$, weighted over regime posterior: $\sum_r P(r \mid \pi_t) \cdot P(\tilde{a} = C \mid \theta_{j,r})$.

101 grid points $\times$ N agents $\times$ 3 regimes = 1212 values. Negligible memory/compute.

6.3.3 Baseline 3: Bayesian + Message Reliability This baseline = Baseline 2 + message-informed regime posterior. It must be a complete prediction model, not just a posterior updater.

Message model (generative): For each sender j and regime $f \in F$:

$$m_{j,t} \mid f_t = f \sim \text{Categorical}(\phi_{j,f}), \quad \phi_{j,f} \sim \text{Dirichlet}(\mathbf{1})$$

Online update: After observing $m_{j,t}$ with regime posterior π_t:

$$\phi_{j,f}^{(v)} \leftarrow \phi_{j,f}^{(v)} + \pi_t(f) \cdot \mathbb{1}[m_{j,t} = v]$$

(Soft Dirichlet count update, weighted by posterior.)

Message-informed regime posterior:

$$\pi_t^{\text{after-msg}}(f) \propto \pi_t^{\text{before-msg}}(f) \cdot \prod_{j \in Z} \text{Cat}(m_{j,t}; \hat{\phi}_{j,f})$$

where $\hat{\phi}_{j,f} = \text{normalised}(\phi_{j,f})$.

Action prediction (complete pipeline): 1. Compute $\pi_t^{\text{after-msg}}$ using message model. 2. Use $\pi_t^{\text{after-msg}}$ to weight regime bins. 3. Predict $P(\tilde{a}_{j,t+1} = C)$ using the ToM-0 cooperativeness grid posteriors from Baseline 2, weighted by the message-updated regime posterior.

Reliability measure: $\rho_j := \max_{f \neq f'} D_{\text{KL}}(\hat{\phi}_{j,f} \| \hat{\phi}_{j,f'})$.

6.3.4 Baseline 3b: Bayesian + Message-as-Intention **Motivation:** Baseline 3 assumes messages encode regime information $(m_{j,t} \mid f_t)$. But emergent communication often encodes intended action, recommendation, or reputation conventions. If trained agents converge to intention-signaling, Baseline 3 looks artificially weak and PLRNN wins are unconvincing.

Message model (generative): For each sender j and intended action $a \in \{C, D\}$:

$$m_{j,t} \mid a_{j,t}^{\text{intended}} = a \sim \text{Categorical}(\psi_{j,a}), \quad \psi_{j,a} \sim \text{Dirichlet}(\mathbf{1})$$

Online update: After observing $m_{j,t}$ and $\tilde{a}_{j,t}$ (executed action as proxy for intended):

$$\psi_{j,a}^{(v)} \leftarrow \psi_{j,a}^{(v)} + (1 - \epsilon_{\text{tremble}}) \cdot \mathbb{1}[\tilde{a}_{j,t} = a] \cdot \mathbb{1}[m_{j,t} = v]$$

(Tremble-corrected soft count: weight by probability that executed action matches intended.)

Action prediction pipeline: 1. Observe $m_{j,t}$. Update belief about j's intended action: $P(a_{j,t}^{\text{intended}} = C \mid m_{j,t}) \propto \text{Cat}(m_{j,t}; \hat{\psi}_{j,C}) \cdot P(a_{j,t}^{\text{intended}} = C)$, where the prior comes from the ToM-0 cooperativeness posterior (Baseline 2). 2. Predict executed action: $P(\tilde{a}_{j,t} = C) = P(a_j^{\text{int}} = C \mid m)(1 - \epsilon) + P(a_j^{\text{int}} = D \mid m)\epsilon$.

What this tests: Whether the PLRNN captures dynamics beyond what a rational listener who correctly interprets messages as action intentions can predict. Combined with Baseline 3 (regime-signaling), we cover the two most common emergent communication functions.

Trust operationalisation: $\text{Trust}_{i \to j,t} := D_{\text{KL}}(\pi_t^{\text{after-msg}} \| \pi_t^{\text{before-msg}})$ — how much j's message shifts the regime posterior.

6.4 Dynamical Analysis Protocol for RQ2

6.4.1 Fixed Points: Non-Autonomous System u_t changes over time; the attractor landscape is parameterized by u.

Compute fixed points at canonical u values:

Label	u value	Interpretation
Certain-competitive	$\delta_{f=0.5}$	Posterior locked on $f = 0.5$
Certain-mixed	$\delta_{f=2.5}$	Posterior locked on $f = 2.5$
Certain-cooperative	$\delta_{f=5.0}$	Posterior locked on $f = 5.0$

Label	u value	Interpretation
Maximum uncertainty	Uniform(F)	No regime information

Additionally: Sample 100 representative u_t values from evaluation data; compute fixed points at each. This gives the "as-experienced" landscape.

6.4.2 Fixed Point Enumeration For each u and activation pattern $D_k = \text{diag}(\mathbb{1}[z > 0])$:

$$z^* = (I - A - WD_k)^{-1}Bu$$

Check consistency: sign of z^* matches D_k.

At $d = 10$: $2^{10} = 1024$ patterns. Fast. **At $d = 15$:** $2^{15} = 32768$. Feasible. **Do not use $d > 15$ for attractor analysis.** If prediction needs higher d, use shPLRNN.

Supplementary for high d: Enumerate only activation patterns visited by inferred trajectories plus small perturbations.

Numerical robustness rules: - If $\kappa(I - A - WD_k) > 10^8$ (condition number), mark the fixed point as unreliable. Use pseudo-inverse to compute it but flag it in results. Do not include unreliable fixed points in attractor counts. - Consistency check: after computing z^*, verify that $\text{sign}(z_j^*) = D_{k,jj}$ for all dimensions. Discard inconsistent solutions. - **Process noise (Q) caveat:** Fixed-point analysis uses the deterministic skeleton ($\eta_t = 0$). Log the learned diagonal of Q. If $\text{tr}(Q)/d > 0.5$, report attractor analysis as "mean-field skeleton only" — stochastic trajectories may not concentrate near fixed points. In this regime, supplement with empirical attractor analysis (cluster long generative rollouts and report cluster centers + dwell times).

6.4.3 Stability, Basins, Interpretation

- **Stability:** Eigenvalues of $J_k = A + WD_k$.
- **Basin volume:** 1000 random $z_0 \sim \mathcal{N}(0, I)$, simulate 200 steps, record convergence.
- **Interpretation:** Pass each z^* through emission model → predicted cooperation rate, message distribution, welfare.
- **Cross-initialisation robustness:** Report attractor counts from top 3 PLRNN fits.
- **Alternative:** Fit single PLRNN with condition as one-hot appended to u_t; analyse condition-slices. Try both; report the more interpretable.

6.4.4 Early Warning (RQ3) — CAUSAL INFERENCE ONLY At each t, compute $|\lambda_{\max}(J_t)|$ using z_t inferred by the **forward-only (causal) encoder**.

Define "cooperation collapse": rolling cooperation (window 10) drops below 0.3 for ≥ 5 rounds.

Baselines: - Rolling cooperation rate crossing 0.5. - MI(message, action) dropping below threshold. - Bayesian trust ($\text{Trust}_{i \to j}$) crossing threshold.

Metric: AUC at lead times $K \in \{1, 3, 5, 10\}$.

6.5 Trust Validation

Step 1: Message responsiveness:

$$R_{i,t} = D_{\text{KL}}(\pi_{A_i}(a \mid o_i, m) \| \pi_{A_i}(a \mid o_i, \tilde{m}))$$

with $\tilde{m}$ from per-sender marginal (in-distribution via dropout training).

Step 2: Correlate PLRNN dimensions with $R_{i,t}$, $\bar{k}_t$, $\mathbb{E}[f_t]$.

Step 3: Partial correlation: show dissociation.

Step 4: Cross-validate against Bayesian trust from Baseline 3.

6.6 Exploitative Signaling

Round t is "exploitative" if: 1. j's message causally increases uncertain receivers' cooperation (vs per-sender marginal perturbation). 2. $r_{j,t} > W_t/N$. 3. Local $\mathrm{MI}(m_j; f_t)$ below threshold.

Call this "exploitative signaling," not "deception."

---

7. Contingency Plans

7.1 Endogenous f Feedback

$$f_{t+1} = \mathrm{clip}(f_t + \alpha(\mathrm{coop_rate}_t - \tau) + \xi_t, f_{\min}, f_{\max})$$

Impact on baselines: Bayesian filter now needs beliefs about others' cooperation → becomes a ToM model. Specify: Bayesian baseline knows the feedback function form but must estimate coop_rate from observations.

7.2 Agent Heterogeneity

Different exploration temperatures or small cooperation bonuses. Second-line.

7.3 Fallback

If make-or-break fails: workshop paper (NeurIPS Cooperative AI, AAMAS) on attractor landscape comparison.

---

8. Implementation Timeline

Week 1–2: Environment Setup

- ☐ Clone marl-emecom. Pin versions (§10). Reproduce single-step baseline results.
- ☐ Rewrite `step()` for multi-step (obs every step, f_t Markov update, proper termination).
- ☐ Fix `observation_space()` → `Box`.
- ☐ Remove observation clamping.
- ☐ Set `gmm_ = False`, `normalize_obs = False`.
- ☐ Build `ObservationWrapper` with history features.
- ☐ Implement trembling-hand noise (env-side).
- ☐ Implement message dropout (wrapper-side).
- ☐ **Implement PPO training loop** (replace myopic REINFORCE for multi-step).
- ☐ Write unit tests.
- ☐ Run regime identifiability audit.

Week 3–4: Agent Training

- ☐ Train conditions 1 and 2 (MVP).
- ☐ Dynamic Richness Checklist.
- ☐ **Decision gate 1:** Escalate if needed. Retrain.
- ☐ Expand to remaining conditions.
- ☐ Generate evaluation datasets.
- ☐ **Consolidate** data into per-(condition, seed) arrays.

Week 5–6: PLRNN + Baselines

- ☐ Implement Bayesian filter (exact HMM forward).
- ☐ Implement Baseline 2 (grid posterior, tremble-corrected).
- ☐ Implement Baseline 3 (Dirichlet-Categorical + ToM-0 prediction pipeline).
- ☐ Implement Baseline 3b (message-as-intention + ToM-0 prediction pipeline).
- ☐ Implement PLRNN-SSM: transition, factored emission, SVAE.
- ☐ Implement forward-only encoder (for RQ3).
- ☐ Implement linear SSM and GRU baselines.
- ☐ Fit all on condition 1 training data.

Week 7: Make-or-Break

- ☐ RQ1b: all models conditioned on π_t, predict at $H = 1, 5, 10, 20$.
- ☐ Oracle-regime control: repeat with $u_t = \delta_{f_t}$.
- ☐ Augmented-posterior robustness: repeat with $\pi_t^{(+k)}$.
- ☐ **Decision gate 2:** PLRNN must beat Baseline 3 at $H \geq 5 \rightarrow$ proceed. Else $\rightarrow$ fallback.

Week 8–9: Dynamical Analysis

- ☐ Fixed points at canonical u values.
- ☐ Basin volumes, stability, interpretation.
- ☐ Cross-initialisation robustness.
- ☐ Early warning (causal encoder, RQ3).
- ☐ Trust validation.

Week 10–11: Writing

Week 12: Revision and Submission

---

9. Reproducibility

- **Seed everything:** `env.seed(s)`, `np.random.seed(s)`, `torch.manual_seed(s)`, `torch.cuda.manual_seed_all(s)`.
- **Deterministic mode:** `torch.backends.cudnn.deterministic = True`, `torch.backends.cudnn.benchmark = False`.
- **Log git commit hash** of env + training code at the start of every run.
- **Save full config** (all hyperparameters, versions, seeds) as JSON per run.

---

10. Pinned Software Versions

Package	Version	Notes
Python	3.9.x	Match marl-emecom
PyTorch	1.13.x or 2.0.x	Verify against repo
PettingZoo	Match repo `requirements.txt`	Pin; do NOT upgrade; 4-tuple API
gymnasium / gym	Match repo	
NumPy	>= 1.21	
SciPy	>= 1.7	
h5py	>= 3.0	For consolidated data
matplotlib	>= 3.5	
scikit-learn	>= 1.0	

First implementation task: `pip install` per repo instructions, then `pip freeze > requirements_locked.txt`. All work uses this locked environment.

---

11. Statistical Implementation Details

MI estimator: Plug-in estimator with Laplace smoothing ($\alpha = 1$). For $\mathrm{MI}(m; f_t)$: bin f_t into regime categories from F; for $\mathrm{MI}(m; a)$: use binary action directly. With $V = 2$ messages and 5 regime bins, the contingency table is small enough that plug-in with Laplace correction is adequate.

Window sizes: - $\mathrm{MI}(m; f_t)$ and $\mathrm{MI}(m; a)$: sliding window of 50 rounds. - Cooperation collapse detection: rolling window of 10 rounds (already specified). - EWMA cooperation $\bar{k}_t$: decay $\lambda = 0.9$ (already specified).

Threshold calibration: All thresholds are calibrated from the training distribution, not hard-coded: - "MI drop" for early warning: 10th percentile of per-session MI distribution from training data. - "Cooperation collapse": rolling cooperation below 0.3 for 5 or more rounds (already specified; 0.3 chosen as midpoint of the [0, 0.5] defection-dominant range with $N = 4$). - Bayesian trust threshold for early warning baseline: 10th percentile of per-session $\mathrm{Trust}_{i \to j}$ distribution from training data.

Small-sample handling: With $N = 4$ agents and $V = 2$, per-round contingency tables are sparse. All MI and conditional probability estimates use the 50-round sliding window to accumulate sufficient counts. Report 95% bootstrap CIs on MI estimates where they appear in tables.

---

12. Key References

- Orzan et al. (2024). Learning in public goods games: the effects of uncertainty and communication. *Neural Computing and Applications.*
- Durstewitz (2017). A state space approach for piecewise-linear recurrent neural networks. *PLOS Comp Bio.*
- Brenner et al. (2022). Tractable dendritic RNNs for reconstructing nonlinear dynamical systems. *ICML.*
- Koppe et al. (2019). Identifying nonlinear dynamical systems via generative recurrent neural networks with applications to fMRI. *PLOS Comp Bio.*
- Behrens et al. (2007). Learning the value of information in an uncertain world. *Nature Neuroscience.*
- Schulman et al. (2017). Proximal Policy Optimization Algorithms. *arXiv.*